LAB 2 · C PROGRAMMING IN THE UNIX ENVIRONMENT

Building the Math Server

A plain-language explanation and a strategic implementation guide

Based on the handout for DV1457/DV1578, Blekinge Institute of Technology (Sai Prashanth Josyula, 1 September 2023)

Part	What you get	Read it when…
A — Understand	What the lab is, an everyday analogy, the step-by-step mechanism of a request, the two math problems and the five OS concepts in plain words, and exactly what earns each grade.	Before you write any code.
B — Implement	A phased roadmap, the folder/Makefile setup, CLI rules, a message protocol, how to parallelise both algorithms with pthreads, and how to build the fork, daemon, poll and epoll servers.	While building, phase by phase.
C — Prove it	A test plan that mirrors the grading scenarios, likely presentation questions, unclear points in the handout, and a final checklist.	Before the presentation and submission.

Academic integrity — read this first

The handout says your code must be your own; you may reuse the provided sequential programs and copy small idioms/snippets, but you must be able to explain **every** part. This guide therefore gives strategy, design and short, generic idioms — not a finished solution. Write the implementation yourselves and make sure both partners understand all of it.

How the advice was checked

The key ideas were compiled and run on Ubuntu 24.04 with gcc 13.3 before being written down: the parallel Gauss–Jordan plan gave results identical to a sequential version; the parallel k-means plan gave identical cluster assignments (50 000 points, k = 9, 4 threads); ThreadSanitizer reported no data races; a minimal fork server answered 20 simultaneous clients with zero zombie processes; the Makefile skeleton built the required layout; and the daemon steps produced a detached process (no terminal, parent PID 1) that stopped cleanly with `kill`.

Contents

1. The lab in one minute . . . 3

2. An everyday analogy: the restaurant . . . 3

3. Step by step: what happens in one session . . . 4

4. The two math problems in plain words . . . 5

4.1 Matrix inversion (Gauss–Jordan elimination) . . . 5

4.2 k-means clustering . . . 5

5. The five Unix concepts, simply . . . 6

6. What is being asked — goals and grading . . . 6

7. Guiding principles . . . 7

8. Phased roadmap . . . 8

9. Project structure and Makefile . . . 8

10. Command-line interface . . . 9

11. Designing the client–server protocol . . . 10

File-naming rules (no overwriting, anywhere) . . . 11

12. Parallelising matrix inversion with pthreads . . . 11

13. Parallelising k-means with pthreads . . . 12

14. Plugging the solvers into the server . . . 13

15. Strategy 1 — fork (required for grade D) . . . 13

16. Daemon mode -d (grade C) . . . 14

17. Strategy 2 — muxbasic with select() or poll() (grade B) . . . 14

18. Strategy 3 — muxscale with epoll() (grade A) . . . 16

19. Robustness checklist . . . 16

20. Test plan that mirrors the grading . . . 17

21. Preparing for the presentation . . . 18

22. Unclear points in the handout — and safe choices . . . 19

23. Final submission checklist . . . 19

Appendix — Glossary and man pages . . . 20

PART A — UNDERSTANDING THE LAB

1. The lab in one minute

You will write **two C programs that talk to each other over a network**. The **server** is a calculator that runs all the time and waits. The **client** is the program a person types into. The person types a request — *invert an 8×8 matrix* or *group these points into 9 clusters* — and the client sends it to the server. The server computes the answer using **several CPU cores at once**, saves it to a file, and **sends the file back**. The client stores it without overwriting earlier answers. Many people can use the server **at the same time**.

The lab is not really about the math — the course gives you working sequential code for both math problems. It is about the **Unix machinery** around it: processes, threads, background services, watching many connections at once, and network sockets.

Item	What the handout requires
Language and tools	C, compiled with `gcc`, built with a single `make` in the project folder.
Platform	Ubuntu's latest LTS release (the provided sequential code was tested on 22.04).
Team	Groups of exactly two. Both present in person; the teacher runs your code and asks questions.
Topics practised	(i) processes and forking, (ii) pthreads, (iii) daemons, (iv) I/O multiplexing, (v) sockets — lectures 6–10.
Starting material	`matrix_inverse.c`, `kmeans.c` (sequential C) and `kmeans-viz.py` (plots k-means output), from Canvas.
Your new files	`server.c`, `client.c`, `matinv-par.c`, `kmeans-par.c` plus whatever helpers you need, in a fixed folder layout.

2. An everyday analogy: the restaurant

Each technical idea in the lab has a simple everyday equivalent. Keep this table in mind; the rest of the guide uses the technical words.

Lab concept	In the restaurant	What it really is
Server program	The kitchen, open all day	A long-running process listening on a TCP port
Client program	A customer who phones in orders	A program that connects to the server's IP address and port
IP address + port	Street address + the right phone line	Which machine, and which program on that machine
Socket	The phone call itself	A two-way stream of bytes between two programs
Command / problem data	The order: "matrix, size 8, fast"	Text (and for k-means a data file) sent over the socket
Result file	The meal, packed in a labelled box	A text file sent back and saved in `client/results/`
fork strategy	Each new customer gets a personal waiter (a clone)	The server creates a child process per client
muxbasic	One waiter walks past every table, asking each in turn	One process uses `select()`/`poll()` over all sockets
muxscale	One waiter with a pager; tables buzz when they need something	One process uses `epoll()`, which reports only ready sockets
pthreads	Several cooks preparing one big dish together	Threads inside one process sharing the computation

Lab concept	In the restaurant	What it really is
Daemon	A kitchen that keeps cooking with nobody at the front desk	A background process detached from any terminal
Zombie process	A waiter who went home but whose timesheet was never signed, so they still appear on the roster	A finished child process whose parent never collected its exit status with `wait()`

3. Step by step: what happens in one session

This is the mechanism you are building, from typing `make` to receiving an answer. The numbers match the diagram.

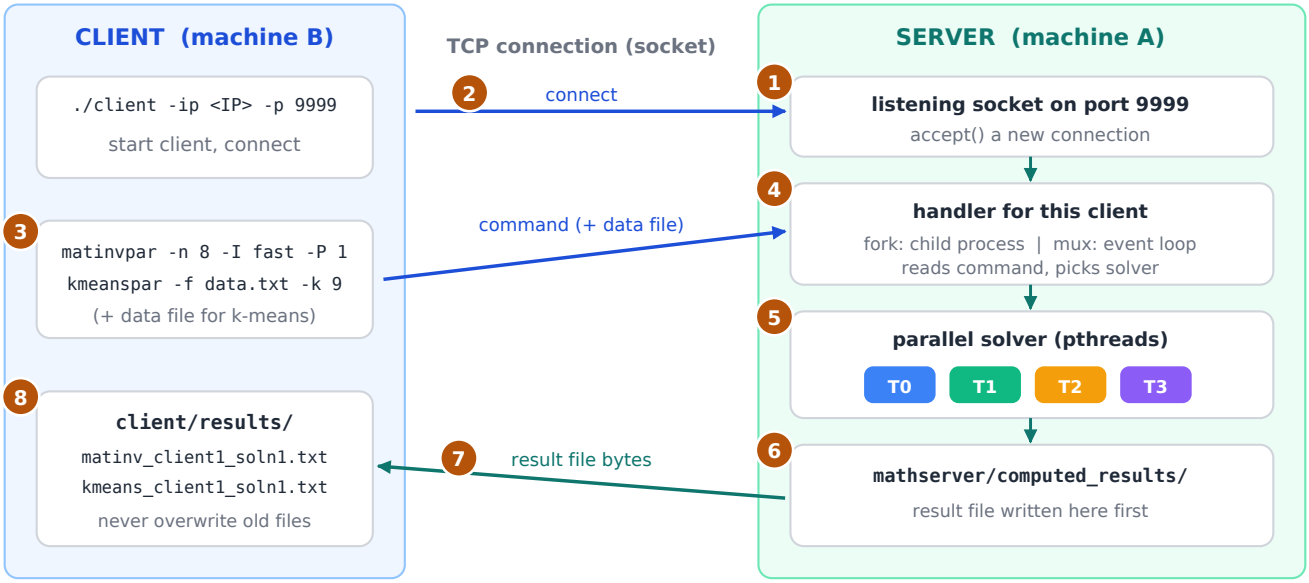


Figure 1 — The journey of one request. Client and server may run on different computers.

1. **Start the server.** `./server -p 9999` (optionally `-s fork|muxbasic|muxscale` and `-d`). It opens a listening socket, prints `Listening for clients...`, and keeps running until Ctrl+C.
2. **Start a client** in another terminal or on another computer: `./client -ip 127.0.0.1 -p 9999`. A TCP connection is made; the server numbers it (client 1, client 2, ...) and prints `Connected with client 1`.
3. **The user types a command**, e.g. `matinvpar -n 8 -I fast -P 1` or `kmeanspar -f kmeans-data.txt -k 9`. The client sends it. For k-means the client also sends the data file, because the server does not have it (FAQ A7). For matrix inversion the server generates the matrix itself from the options.
4. **A handler takes over.** With *fork* it is a dedicated child process. With *mux* it is the single event loop that serves every client.
5. **The handler runs the parallel solver** — matrix inversion or k-means — split across several pthreads.
6. **The solver writes a result file** in `mathserver/computed_results/` with a unique name, e.g. `matinv_client1_soln1.txt`.
7. **The server sends the file** back over the socket and prints `Sending solution: matinv_client1_soln1.txt`.
8. **The client saves it** in `client/results/` without overwriting older files, prints `Received the solution: …`, and asks for the next command. Meanwhile other clients can connect and be served at the same time.

The goal in one sentence

Make a small but realistic network service: **correct** (same output as the sequential programs), **parallel** (pthreads inside the math), **concurrent** (many clients at once), **well-behaved as a Unix process** (no zombies, optional daemon mode, proper exit codes), and **cleanly organised** (fixed folder layout, one `make`).

4. The two math problems in plain words

4.1 Matrix inversion (Gauss–Jordan elimination)

Think of a matrix as a recipe that scrambles a list of numbers. Its **inverse** is the recipe that unscrambles them: apply one after the other and you get back what you started with. In matrix terms $A \times A^{-1} = I$, where I (the identity matrix) is the do-nothing matrix: 1s on the diagonal and 0s elsewhere.

Gauss–Jordan finds the inverse by tidying rows. Write A next to I. Apply the same row operations to both until the left side becomes I. The right side is then A^{-1}. Only two moves are needed: *divide a row by a number*, and *subtract a multiple of one row from another row*.

	Handout example (checked exactly with fractions)
Input A	[3 6 9] [2 8 15] [7 3 8]
Step for pivot 1	R1 → R1/3, then R2 → R2 − 2·R1 and R3 → R3 − 7·R1 (zero the rest of column 1)
Step for pivot 2	R2 → R2/4, then R1 → R1 − 2·R2 and R3 → R3 + 11·R2
Step for pivot 3	Same idea for column 3
Result A^{-1}	[19/141 −7/47 6/47] [89/141 −13/47 −9/47] [−50/141 11/47 4/47]

Why this is a good candidate for threads

For one pivot, updating each of the other rows is **independent** — row 5 does not need row 7's new values. So the rows can be shared out among threads. But the pivots must be processed **in order** (pivot 2 needs the finished result of pivot 1). For n = 2048 that is roughly $2 \cdot n^3 \approx 17$ billion multiply-subtract operations, so using several cores makes a visible difference.

4.2 k-means clustering

Imagine a field full of people, and you want them in **k groups**, each with a meeting point:

1. Pick k people at random; their positions are the first meeting points (**centroids**).
2. Everyone walks to the **nearest** meeting point (assignment step).
3. Move each meeting point to the **average position** of its group (update step).
4. Repeat steps 2–3 until **nobody changes group**. The answer is the group number of every point.

Why this is a good candidate for threads

Step 2 is independent for every point: thread 0 takes points 0–9 999, thread 1 takes 10 000–19 999, and so on. Step 3 needs everyone's results combined, so each thread keeps its **own subtotals** (sum of x, sum of y, count per cluster) and one thread adds them up after all threads finish.

5. The five Unix concepts, simply

Concept	Plain explanation	Where it appears in the lab
Processes and `fork()`	`fork()` copies the running program. Both copies continue from the same line; `fork()` returns 0 in the child and the child's PID in the parent. Each has its own memory.	fork strategy (one child per client); daemon mode.
pthreads	Several threads of execution inside **one** process, sharing its memory. Cheap to create, but they must be coordinated (`pthread_join`, barriers, mutexes) to avoid stepping on each other.	Inside both solvers (`matinv-par.c`, `kmeans-par.c`).
Daemons	A background service with no terminal. Made by forking, letting the parent exit, calling `setsid()`, and detaching stdin/stdout/stderr.	The `-d` option (grade C and up).
I/O multiplexing	One thread watches many connections and reacts to whichever is ready, instead of one process per connection.	muxbasic (`select`/`poll`, grade B), muxscale (`epoll`, grade A).
Sockets	Endpoints of a network connection. Server: `socket → bind → listen → accept`. Client: `socket → connect`. Both: `send`/`recv`, then `close`.	All client–server communication.
Zombies (rule 1.4)	When a child process ends, its exit status waits for the parent to collect it with `wait()`/`waitpid()`. Until then it is a zombie (`Z` / `<defunct>` in `ps`).	The server must never leave zombies, whatever the load.

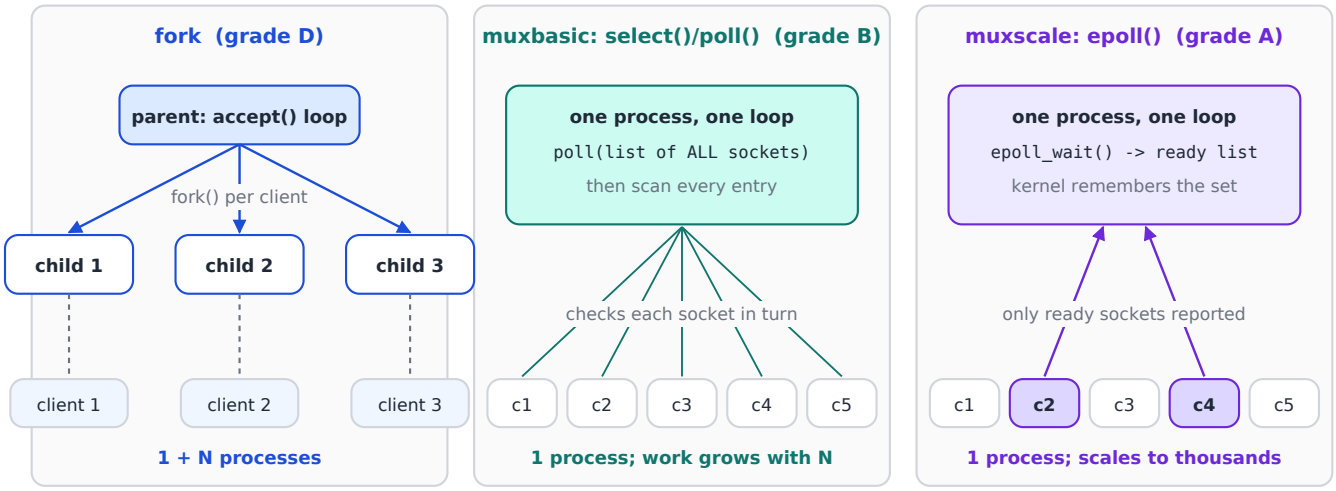


Figure 2 — The three ways your server can serve many clients.

6. What is being asked — goals and grading

Some rules apply to **every** grade. Breaking the first one ends the evaluation immediately.

Non-negotiable (all grades)

Folder structure exactly as in section 1.1. Otherwise: grade F, no evaluation, no feedback, and a lost submission attempt.

`make` **in the project folder builds both server and client** and places the executables in the project folder; object files go in `mathserver/objects/`.

Command-line options of section 1.5, including: print help and exit with return value **3** when an option is not supported.

Concurrent clients (1.3); the server keeps running until **Ctrl+C**; **no zombies** (1.4).

Results are files sent back over the socket, in the **same format** as the sequential programs' output, and never overwritten.

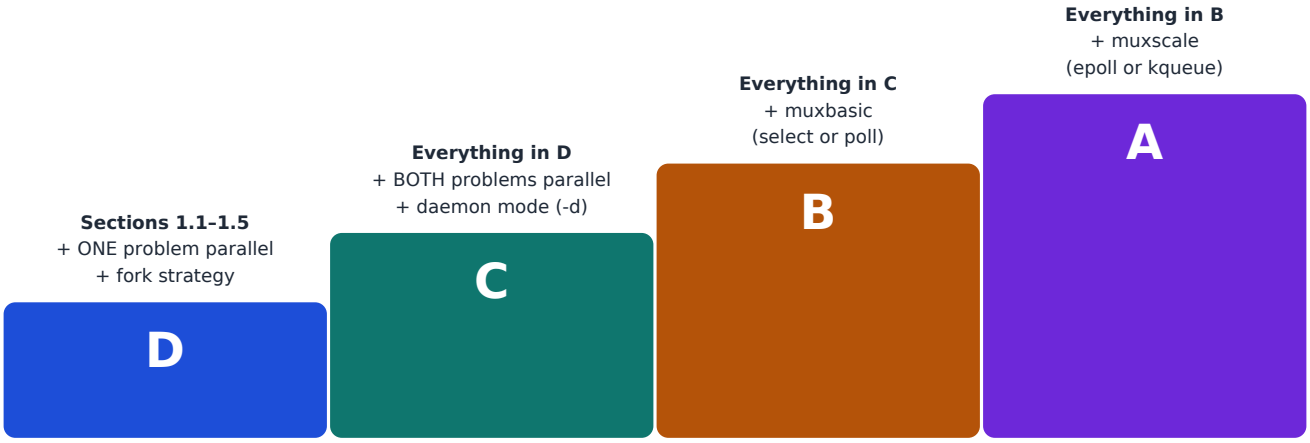


Figure 3 — The grade ladder. Each grade includes everything below it.

- **Code quality matters:** meeting a grade's requirements with poor code can cost one grade. Grade E = grade-D features with poor code quality.
- **No speedup target** (FAQ A5): 2–5× faster than sequential is fine. But you must parallelise the **algorithm itself**. Parallelising only the matrix initialisation is not accepted.
- **At the presentation** you must explain your strategy, including its advantages and drawbacks, and the teacher runs the tests in section 20 of this guide.

PART B — IMPLEMENTATION STRATEGY

7. Guiding principles

- **Always have something that works.** Build in layers; each phase ends with a runnable, demonstrable program. Commit to git at every milestone so you can roll back.
- **Lock the folder structure on day one.** It is the only mistake that can earn an F without evaluation.
- **Correctness before speed.** After every change to a solver, `diff` its output against the sequential program's output.
- **Keep the network and the math separate.** Socket code knows nothing about matrices; solvers know nothing about sockets. They meet through two small functions.
- **Make solvers re-entrant** (no global state). The mux strategies run everything in one process, so this is needed there; it does no harm with fork.
- **Split the work, share the understanding.** Each partner leads one track, but both must be able to explain every line (plagiarism rule and presentation).
- **Climb the ladder in order:** D → C → B → A. A polished C beats a broken B; the order also means you always have a gradable version.

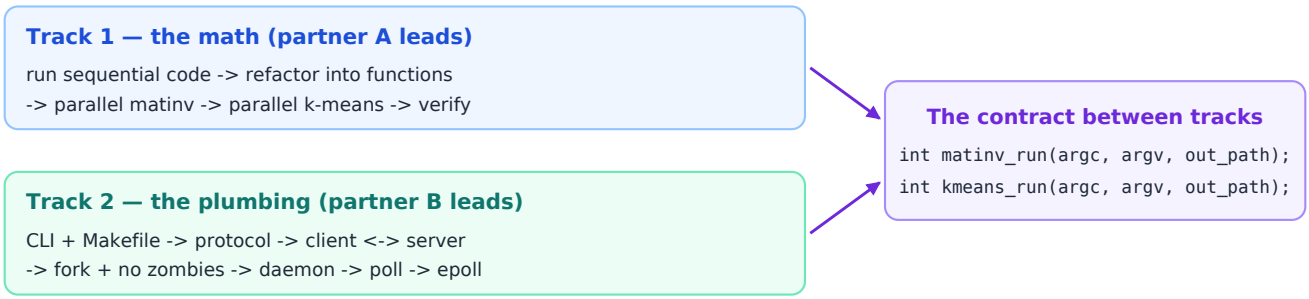


Figure 4 — Two parallel work tracks that meet at a small, agreed interface.

8. Phased roadmap

#	Phase	Done when…	Unlocks
0	**Set up and observe.** Ubuntu LTS machine/VM, `gcc`, `make`, `git`, Python 3 for `kmeans-viz.py`. Build and run both sequential programs; save their outputs as **reference files**.	Both programs run; reference outputs saved for several inputs.	—
1	**Skeleton.** Exact folder tree, Makefile, empty `server.c` and `client.c`, full option parsing (`-h`, unsupported option → usage + exit 3).	`make` builds both; `./server -x; echo $?` prints 3.	—
2	**Solvers as functions.** Turn each sequential `main` into `matinv_run(...)` / `kmeans_run(...)` that write to a given file path; add `-f` and `-k` to k-means.	Output files are byte-identical to the sequential programs.	—
3	**Parallel matrix inversion** with pthreads (section 12).	`diff` identical for many sizes and thread counts; measured speedup.	D (1 of 2 problems)
4	**One client end to end.** Protocol, send command, receive and save the file.	One client gets correct, uniquely named files.	—
5	**fork strategy** + zombie reaping + client numbering.	3+ clients at once; `ps` shows no `Z`.	**Grade D**
6	**Parallel k-means** + data-file upload (section 13).	Same output as sequential; plot looks right.	—
7	**Daemon mode** `-d` (section 16).	Prints PID; survives closing the terminal; `kill PID` stops it.	**Grade C**
8	**muxbasic** with `poll()` or `select()` (section 17).	Many clients served by **one** server process.	**Grade B**
9	**muxscale** with `epoll()` (section 18).	Hundreds to thousands of scripted clients succeed.	**Grade A**
10	**Harden and rehearse** (sections 19–23).	Full test plan passes; both can explain everything.	Submit

9. Project structure and Makefile

Create this tree first. The names listed in the caption are required by the handout; the other files are suggestions for splitting the code cleanly.

Recommended layout. Required by the handout: Makefile, mathserver/, include/, objects/, src/, server.c, computed_results/, client/, client.c, results/

```
project/
├── Makefile                    <- 'make' here builds everything
├── server                      <- built executable (project folder)
├── mathserver/
│   ├── include/                <- ALL server headers (.h)
│   │   ├── protocol.h          message types + header struct
│   │   ├── net_util.h          send_all / recv_all / send_msg ...
│   │   ├── matinv_par.h        int matinv_run(int, char **, const char *);
│   │   ├── kmeans_par.h        int kmeans_run(int, char **, const char *);
│   │   └── strategies.h        run_fork / run_muxbasic / run_muxscale
│   ├── objects/                <- .o files created by make
│   ├── src/                    <- ALL server sources (.c)
│   │   ├── server.c            main: options, daemon, choose strategy
│   │   ├── matinv-par.c        parallel matrix inversion
│   │   ├── kmeans-par.c        parallel k-means
│   │   ├── net_util.c  strategy_fork.c  strategy_mux.c  daemon.c ...
│   └── computed_results/       <- server writes result files here
└── client/
    ├── client.c                (+ a copy of protocol.h if you like)
    └── results/                <- client saves received files here
```

Trap found while testing: the client executable's name

The handout says `make` places the executables in the project folder, and the examples run `./client`. But the project folder must also contain a **directory** called `client/`, and a file and a directory cannot have the same name. Linking to `client` fails with `cannot open output file client: Is a directory`.

Safe options: (a) build the client as `client/client` and run it from inside `client/` — this matches the `./client -ip …` example and makes `results/` a simple relative path; or (b) give it another name in the project folder (e.g. `mathclient`). **Ask your TA which they expect** and write the choice in your README.

Makefile skeleton (tested). In the real file, recipe lines must start with a TAB, not spaces.

```
CC      := gcc
CFLAGS  := -Wall -Wextra -O2 -Imathserver/include
LDLIBS  := -lpthread

SRV_SRC := $(wildcard mathserver/src/*.c)
SRV_OBJ := $(patsubst mathserver/src/%.c,mathserver/objects/%.o,$(SRV_SRC))
SERVER  := server
CLIENT  := client/client          # see the warning box above

all: $(SERVER) $(CLIENT)

$(SERVER): $(SRV_OBJ)
    $(CC) $(CFLAGS) -o $@ $^ $(LDLIBS)

mathserver/objects/%.o: mathserver/src/%.c | mathserver/objects
    $(CC) $(CFLAGS) -c $< -o $@

mathserver/objects:
    mkdir -p $@

$(CLIENT): client/client.c
    $(CC) -Wall -Wextra -O2 -o $@ $^

clean:
    rm -f $(SERVER) $(CLIENT) mathserver/objects/*.o

.PHONY: all clean
```

- Only **one** `main()` may end up in the server binary. If you keep standalone versions of the solvers for testing, guard their `main` with `#ifdef STANDALONE` and build them with a separate target.
- Create `computed_results/` and `results/` from your programs at start-up too (`mkdir()` and ignore `EEXIST`), in case they are missing.

10. Command-line interface

Program	Options	Behaviour to get right
`server`	`-h -p port -d -s fork|muxbasic|muxscale`	Options override your defaults (e.g. port 9999, strategy fork). Unknown option, missing value, or a strategy you did not implement → print usage, **return 3**. `-h` → print help (exit 0 is conventional).
`client`	`-ip address -p port`	`-ip` is a multi-letter option with a single dash; plain `getopt()` cannot handle it (see below).
`matinvpar` command	All options of the sequential `matrix_inverse.c` (e.g. `-n`, `-I`, `-P` and the rest listed in its usage text)	Always write the result to a file, whether or not `-P` asks for printing (FAQ A11). Same number format as the sequential output.
`kmeanspar` command	At least `-f file` and `-k clusters`	The file named by `-f` lives on the **client**; the client uploads it and the server saves its own copy.

Client option parsing idiom

```
/* getopt(argc, argv, "i:p:") reads "-ip 127.0.0.1" as option -i with value "p"
   (tested). getopt_long_only accepts single-dash long options: */
static struct option opts[] = {
    { "ip", required_argument, NULL, 'i' },
    { "p",  required_argument, NULL, 'p' },
    { "h",  no_argument,       NULL, 'h' },
    { NULL, 0, NULL, 0 }
};
while ((c = getopt_long_only(argc, argv, "", opts, NULL)) != -1) { ... }
```

Parsing the client's command on the server

Split the received text into an argv array (e.g. `strtok_r` on spaces), with the command name as `argv[0]`, and pass it to the solver's own option parser. **Reset getopt before every new parse**: on glibc set `optind = 0` (a full reset; POSIX says 1). GNU getopt also reorders argv while it scans, so always parse a fresh copy. Validate numbers with `strtol` and an end pointer, and reject nonsense (n ≤ 0, k ≤ 0, k larger than the number of points) with an error message instead of crashing.

11. Designing the client–server protocol

TCP delivers a **stream of bytes**, not messages. One `send()` of 100 bytes may arrive as 60 + 40 bytes, and two sends may arrive glued together. So you must decide how the receiver knows where a message ends. The simplest robust answer is a **length prefix**:

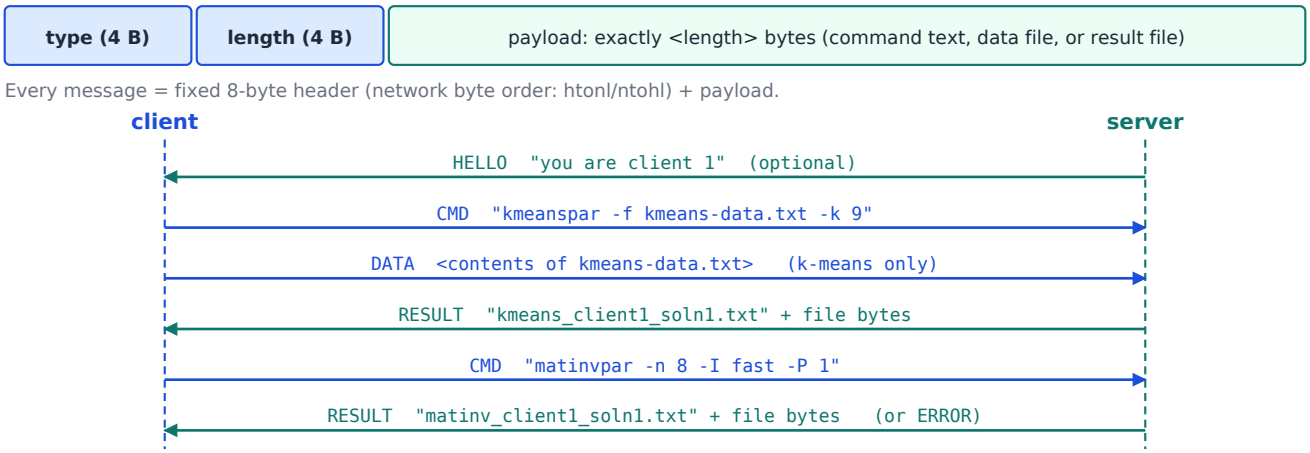


Figure 5 — A length-prefixed message and a typical conversation.

Blocking helpers (tested) — good for the client and the fork strategy

```
#include <stdint.h>
#include <errno.h>
#include <arpa/inet.h>
#include <sys/socket.h>

/* send() and recv() may move fewer bytes than asked: loop until done. */
int send_all(int fd, const void *buf, size_t len) {
    const char *p = buf;
    while (len > 0) {
        ssize_t n = send(fd, p, len, MSG_NOSIGNAL);  /* no SIGPIPE if peer left */
        if (n < 0) { if (errno == EINTR) continue; return -1; }
        p += n; len -= (size_t)n;
    }
    return 0;
}
int recv_all(int fd, void *buf, size_t len) {
    char *p = buf;
    while (len > 0) {
        ssize_t n = recv(fd, p, len, 0);
        if (n == 0) return -1;                      /* peer closed */
        if (n < 0) { if (errno == EINTR) continue; return -1; }
        p += n; len -= (size_t)n;
    }
    return 0;
}
struct msg_hdr { uint32_t type; uint32_t len; };    /* both sent with htonl() */
```

File-naming rules (no overwriting, anywhere)

- **Count per client and per problem type.** The handout example produces `matinv_client1_soln1.txt`, `matinv_client1_soln2.txt`, then `kmeans_client1_soln1.txt` — so k-means has its own counter.
- **Server side:** write to `computed_results/<type>_client<C>_soln<S>.txt`; save uploaded k-means input as e.g. `computed_results/kmeans_client<C>_data<S>.txt` and point the solver's `-f` at that copy.
- **Client side:** save to `results/` under the received name. Open with `O_CREAT | O_EXCL`; if the file exists (for example after the server restarted and numbering began again at client 1), add a suffix. This guarantees the rule "may not overwrite a previously received file".
- Result files can be large (tens of megabytes of text for n = 2048), so send them in chunks or as one length-prefixed payload — never assume one `recv()` gets it all.

12. Parallelising matrix inversion with pthreads

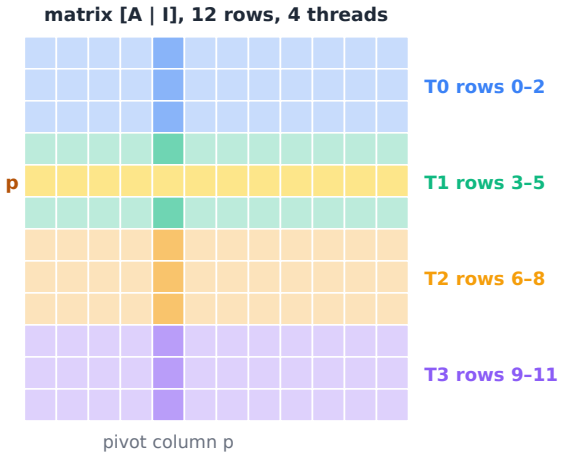


Figure 6 — Rows are divided among threads; the pivot loop stays in order, with two barriers per pivot.

Sketch of the thread function (the tested version matched the sequential result exactly)

```
/* Per job: n, A, I (malloc'd), T threads, pthread_barrier_t bar (count = T).
   Thread t owns rows [t*n/T, (t+1)*n/T). */
for (p = 0; p < n; p++) {
    if (t == 0)                                  /* 1. normalise pivot row  */
        divide row p of A and of I by A[p][p];
    pthread_barrier_wait(&bar);                  /* 2. pivot row is ready   */
    for (r = my_first; r < my_last; r++) {       /* 3. eliminate own rows   */
        if (r == p) continue;
        m = A[r][p];                             /*    read BEFORE updating */
        for (c = 0; c < n; c++) { A[r][c] -= m * A[p][c];  I[r][c] -= m * I[p][c]; }
    }
    pthread_barrier_wait(&bar);                  /* 4. all done with pivot p */
}
```

Design choice	Simple version	Better version (recommended)
Thread lifetime	Create T threads and join them for **each** pivot	Create T threads **once**; synchronise with a `pthread_barrier_t`
Cost	n × T thread creations (thousands for n = 2048)	2 barrier waits per pivot
Explain at presentation	Easy to reason about	Less overhead; shows you understand barriers

- Number of threads: `sysconf(_SC_NPROCESSORS_ONLN)`, and never more than n.
- **Barrier 1** stops threads reading the pivot row while it is being divided. **Barrier 2** stops a fast thread from starting pivot p+1 while others still use pivot p.
- Keep the algorithm the same as the sequential code (same operations, same order per element). Then results are identical, and `diff` is a perfect test.
- Allocate matrices per job with `malloc` (not huge global arrays). Free them when the job ends — the server runs forever.
- Test with sizes 1, 2, 3, 7, 8, 64, 512, 2048, both init modes, and odd thread counts. Build once with `-fsanitize=thread` to catch data races. Time it with `time`.

13. Parallelising k-means with pthreads

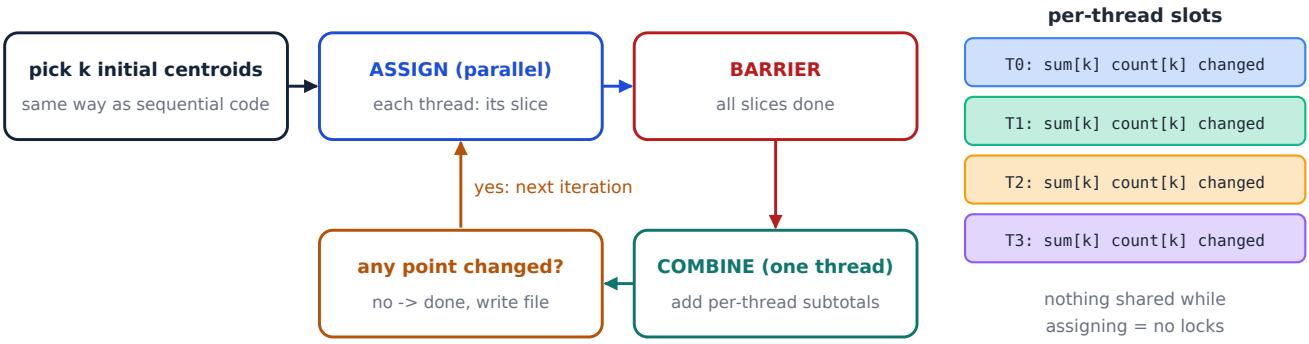


Figure 7 — One k-means iteration with T threads: parallel assignment, then a single combine step.

- **Never** let threads update one shared `changed` flag or one shared sum array without protection. Per-thread slots are simpler and faster than a mutex.
- Pick the initial centroids **exactly as the sequential program does**, and keep its iteration limit and empty-cluster rule. Otherwise results will differ and you lose `diff` as a test.
- Floating-point addition depends on order. Combining subtotals in a fixed thread order keeps results reproducible. If a value differs in the last digit, compare cluster assignments, not raw text.

- Size the arrays from the input file (count points first) instead of a fixed `#define`, and check that k ≤ number of points.
- Check visually with `python3 kmeans-viz.py` on your parallel output.

Sketch of the thread function (tested: identical assignments and iteration count to sequential)

```
/* Thread t owns points [t*N/T, (t+1)*N/T). Slots indexed by t => no locks. */
for (;;) {
    clear my sum_x[t][*], sum_y[t][*], count[t][*]; changed[t] = 0;
    for each of my points i:
        c = nearest_centroid(point[i]);
        if (c != cluster[i]) { cluster[i] = c; changed[t] = 1; }
        sum_x[t][c] += x;  sum_y[t][c] += y;  count[t][c]++;
    pthread_barrier_wait(&bar);
    if (t == 0) {                       /* combine in a FIXED order: t = 0..T-1 */
        done = no thread changed anything;
        if (!done) new centroid[c] = total sum / total count  (skip empty clusters);
    }
    pthread_barrier_wait(&bar);
    if (done) break;                    /* every thread sees the same decision  */
}
```

14. Plugging the solvers into the server

Give each solver a function that the server calls directly, for example:

```
/* Returns 0 on success; on failure writes a short message into err. */
int matinv_run(int argc, char **argv, const char *out_path, char *err, size_t errlen);
int kmeans_run(int argc, char **argv, const char *out_path, char *err, size_t errlen);
```

- **Why functions and not `fork()+exec()` of separate binaries?** In the mux strategies the server must stay a single process (grading test 3). Calling functions works the same way for all three strategies.
- **Re-entrancy checklist:** no global matrices, sizes or flags (keep them in a per-job struct); no `static` buffers; reset getopt for each call; `rand()` is shared state — if two jobs can run at once in one process, use `rand_r()` with a per-job seed (and keep the sequential seed if outputs must match).
- **Output:** whatever the sequential code prints as its result goes into the file at `out_path`. Use the same `printf` formats, so the files match.

15. Strategy 1 — fork (required for grade D)

Shape of the fork server (tested with 20 simultaneous clients)

```
listen_fd = socket(AF_INET, SOCK_STREAM, 0);
setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &one, sizeof one);
bind(listen_fd, {AF_INET, INADDR_ANY, htons(port)});  listen(listen_fd, SOMAXCONN);
install SIGCHLD handler (below);  signal(SIGPIPE, SIG_IGN);
for (;;) {
    fd = accept(listen_fd, ...);   if (fd < 0) { if (errno == EINTR) continue; ... }
    client_no++;                   fflush(stdout);        /* before fork! */
    pid = fork();
    if (pid == 0) { close(listen_fd); serve_client(fd, client_no); close(fd); _exit(0); }
    close(fd);                     /* parent: the child owns this connection now */
}
```

Zombie prevention (tested: 0 zombies after 20 clients)

```
static void on_sigchld(int sig) {
    (void)sig;
    int saved = errno;                          /* waitpid may change errno */
    while (waitpid(-1, NULL, WNOHANG) > 0) { }  /* reap ALL finished children */
    errno = saved;
}
struct sigaction sa = {0};
sa.sa_handler = on_sigchld;  sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;    /* accept() resumes instead of failing with EINTR */
sigaction(SIGCHLD, &sa, NULL);
```

- Alternative: `signal(SIGCHLD, SIG_IGN)` — on Linux the kernel then reaps children automatically. The handler version lets you log when children exit; be ready to explain either.
- **Use a loop in the handler:** several children can finish while only one SIGCHLD is delivered.
- `fflush(stdout)` **before** `fork()`, or text still in the buffer is printed twice (once by each process).
- The parent must `close(fd)` after forking, and the child must `close(listen_fd)`. Otherwise file descriptors leak and clients may never see the connection close.
- The child must end with `_exit()` / `exit()` and must never fall back into the accept loop.

16. Daemon mode -d (grade C)

Daemonising steps (tested: PID printed, TTY '?', parent PID 1, stops with kill)

```
fflush(stdout);
pid = fork();
if (pid > 0) { printf("Server running as daemon, PID %d\n", pid); exit(0); } /* parent */
setsid();                                       /* new session, no terminal      */
umask(022);
/* chdir("/") is traditional; if you do it, use absolute paths for
   computed_results (resolve them with realpath() BEFORE daemonising).     */
fd = open("/dev/null", O_RDWR);                 /* detach standard streams    */
dup2(fd, 0); dup2(fd, 1); dup2(fd, 2);
openlog("mathserver", LOG_PID, LOG_DAEMON);     /* log with syslog() instead  */
/* ... continue into the chosen strategy's loop ... */
```

- The handout requires the PID to be printed. With a single fork the parent can print the child's PID. If you add the optional second fork, print the PID of the **final** process.
- A daemon has no terminal, so **Ctrl+C cannot reach it**. Stop it with `kill <PID>` (that is why the PID is printed). Handle SIGTERM if you want a clean shutdown.
- Verify with `ps -o pid,ppid,sid,tty,stat,cmd -p <PID>`: TTY shows `?`. Close the terminal and check that clients still work.
- Messages like `Connected with client 1` now go to `/dev/null` unless you send them to syslog (`journalctl -t mathserver`) or a log file.

17. Strategy 2 — muxbasic with select() or poll() (grade B)

One process and one loop: **wait until any socket is ready, do a little work for it, go back to waiting.** Because nothing may block the loop, each client needs a small **state record** that remembers how far its conversation has got.

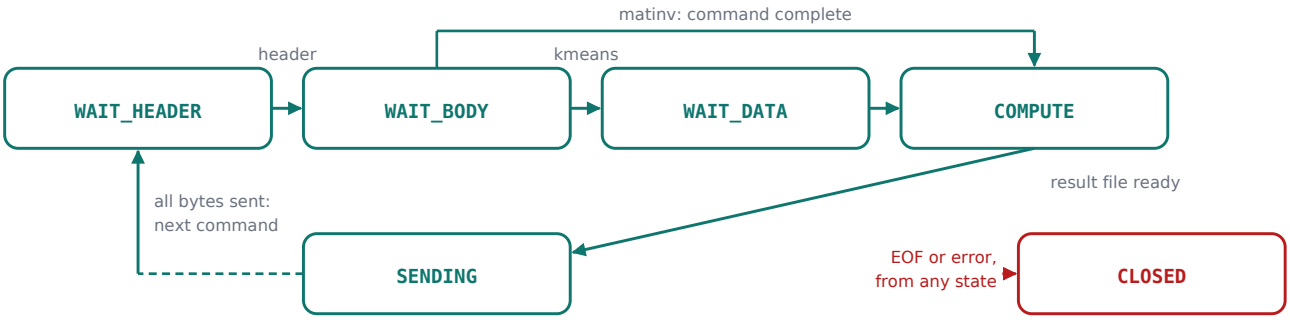


Figure 8 — Per-client state machine used by both multiplexing strategies.

Loop outline

```
struct client {
    int fd, number, matinv_count, kmeans_count, state;
    char *in;  size_t in_len, in_need;        /* bytes collected / expected  */
    char *out; size_t out_len, out_sent;      /* pending reply               */
};
make listen_fd non-blocking; add it to pollfds
for (;;) {
    poll(pollfds, count, -1);
    if (listen_fd readable)  accept, set O_NONBLOCK, allocate struct client, add POLLIN
    for each client fd with events:
        POLLIN : recv() what is available into c->in; on EAGAIN stop; 0 => close
                 when a full message has arrived => advance the state machine
        POLLOUT: send() from c->out + c->out_sent; on EAGAIN stop;
                 when everything is sent => drop POLLOUT, back to WAIT_HEADER
        POLLHUP/POLLERR: close and free the client
}
```

- Make client sockets non-blocking (`fcntl(fd, F_SETFL, O_NONBLOCK)`) and treat `EAGAIN`/`EWOULDBLOCK` as "come back later", keeping partial data in the client's buffers.
- Ask for `POLLOUT` **only while there is something to send**. A socket is almost always writable, so leaving it on makes the loop spin at 100 % CPU.
- `select()` handles at most `FD_SETSIZE` (1024) descriptors and its sets must be rebuilt every time round. `poll()` has no such limit — a good choice for muxbasic.

The compute question — decide, explain, and confirm with your TA

Option A (simple, matches FAQ A8): when a command is complete, call the solver directly. The solver still uses its pthreads, but other clients wait until it finishes. They are not lost; the kernel queues their data.

Option B (more responsive): hand the job to a worker thread. When it finishes, the worker writes one byte to a pipe (or eventfd) that is in the poll set; the loop then switches that client to SENDING. This needs re-entrant solvers and a mutex-protected job queue. It is still one process.

Grading test 2 starts a new client while another computes. Option A passes if the new client can connect and is answered afterwards; Option B answers it right away. Be ready to explain the trade-off.

Check the process model: `ps -ef | grep server` should show **one** server process; `ps -L -p <PID>` shows the solver threads while a job is running.

18. Strategy 3 — muxscale with epoll() (grade A)

Same state machine, different "waiter". With `poll()` you hand the kernel the whole list on every call and then scan all of it. With **epoll** the kernel keeps the list and returns **only the sockets that are ready** — the cost depends on activity, not on how many clients are connected.

Loop outline

```
ep = epoll_create1(0);
ev.events = EPOLLIN; ev.data.ptr = NULL /* or a marker */;  epoll_ctl(ep, EPOLL_CTL_ADD, listen_fd, &ev);
for (;;) {
    n = epoll_wait(ep, events, MAX_EVENTS, -1);
    for (i = 0; i < n; i++) {
        if (it is the listen socket) { accept in a loop until EAGAIN; ADD each with data.ptr = client }
        else { struct client *c = events[i].data.ptr;
               handle_readable(c) / handle_writable(c);   /* SAME code as muxbasic */
               EPOLL_CTL_MOD to add/remove EPOLLOUT; on close: EPOLL_CTL_DEL, close, free }
    }
}
```

- **Reuse code:** write `handle_readable()` / `handle_writable()` once and call them from both mux loops. Only the waiting mechanism differs — a clear code-quality win.
- **Level-triggered** (the default) is easiest. Edge-triggered (`EPOLLET`) is faster, but you must read and write until `EAGAIN` every time, or you will never be woken again.
- **For thousands of clients:** raise the open-file limit (`ulimit -n 65535`, or `setrlimit` in the program; the default is often 1024), use `listen(fd, SOMAXCONN)`, and free each client's memory when it disconnects.
- `kqueue()` is the BSD/macOS equivalent; on Ubuntu use epoll.

	fork	muxbasic (poll/select)	muxscale (epoll)
Processes	1 + one per client	1	1
Memory per client	High (a whole process)	Small struct	Small struct
Isolation	A crash kills only one client's child	A crash kills the server	A crash kills the server
Code complexity	Lowest (blocking code)	Higher (non-blocking + state machine)	Like muxbasic, plus epoll set management
Scalability	Hundreds	select ≤ 1024 fds; poll is O(N) per call	Tens of thousands; O(ready)
Best for	Few, long, independent sessions	Moderate numbers of clients	Many mostly idle connections

19. Robustness checklist

Do this	Because
`setsockopt(SO_REUSEADDR)` before `bind()`	You can restart the server right away without "Address already in use".
Bind to `INADDR_ANY`	Clients on other computers must be able to connect. Find the IP with `hostname -I`; open the port if a firewall is on (`sudo ufw allow 9999/tcp`).
Ignore SIGPIPE (`signal(SIGPIPE, SIG_IGN)` or `MSG_NOSIGNAL`)	Otherwise a client that disconnects mid-transfer kills the server.
Check every system-call return value; `perror` with context	Silent failures are the hardest bugs to find, and checking is part of code quality.
Retry on `EINTR`	Signals such as SIGCHLD interrupt blocking calls.

Do this	Because
Treat `recv() == 0` as "client left"	Clean up its state and files; keep serving everyone else.
Validate every command	Bad input should produce an ERROR message to that client, not a crash.
Free memory and close files after every job	The server runs indefinitely. Check with `valgrind --leak-check=full`.
Zero warnings with `-Wall -Wextra`	Warnings often point at real bugs, and graders notice them.
Client stops cleanly at end of input (EOF on stdin)	Lets you drive tests from scripts (`printf … | ./client …`).

PART C — PROVE IT, PRESENT IT, SUBMIT IT

20. Test plan that mirrors the grading

#	Test	How	Expected
1	Build	`make clean && make`	Executables built, `.o` files in `mathserver/objects/`, no warnings
2	Options	`./server -h`; `./server -z; echo $?`; `./server -s nope; echo $?`	Help text; 3; 3
3	Matrix correctness	`matinvpar -n 8 -I fast -P 1` vs sequential `./matinv -n 8 -I fast -P 1`	Same values and same format
4	k-means correctness	`kmeanspar -f kmeans-data.txt -k 9` vs sequential output for the same k	Same assignments; plot looks right
5	No overwrite	Send the same command three times; restart server; send again	soln1, soln2, soln3 …; nothing replaced
6	Concurrency (grading test 2)	Client 1: `matinvpar -n 2048 -I rand -P 0`; at once start client 2 with a small job	Client 2 communicates as intended
7	Process model (grading test 3)	3 clients connected; `pstree -p <PID>` or `ps -ef | grep server`	fork: 1 + 3 processes. mux: 1 process
8	Zombies	Connect and quit 50 clients; `ps -eo pid,ppid,stat,cmd | grep -w Z`	Nothing listed
9	Daemon	`./server -d -p 9999`; close terminal; run a client; `kill <PID>`	PID printed, TTY `?`, works, stops
10	Scale (muxscale)	Script launching 500–1000 clients in background	All get correct files; server stays up
11	Two computers	Server on machine A, client on machine B with A's IP	Works as on one machine
12	Abuse	Kill a client mid-transfer; send garbage; huge n; then Ctrl+C the server	Server survives; exits cleanly on Ctrl+C
13	Sanitisers	Rebuild with `-fsanitize=address` and, separately, `-fsanitize=thread`; rerun 3–7	No reports
14	Speedup	`time` sequential vs parallel for n = 1024 and 2048	Faster; 2–5× is fine (FAQ A5)

Load-test idea (requires the client to exit at end of input)

```
#!/bin/bash
# many_clients.sh N  -- start N scripted clients at once, then count the results
N=${1:-200}
for i in $(seq 1 "$N"); do
  printf 'matinvpar -n 64 -I rand -P 0\n' | ./client -ip 127.0.0.1 -p 9999 > /dev/null &
done
wait
ls client/results | wc -l      # expect N new files (adjust the path to your layout)
```

21. Preparing for the presentation

Both partners should be able to answer these without looking at notes. The short answer pointers show where to look.

Likely question	Answer pointers
Clients are already forked — why use threads for the math?	FAQ A6: with only one or two clients, threads are the only way to use all the cores.
What exactly did you parallelise in Gauss–Jordan? Why not the pivot loop?	The elimination of rows inside one pivot step; each pivot needs the previous one's result.
Why two barriers per pivot? What breaks without them?	Section 12: reading a half-normalised pivot row; starting pivot p+1 early.
How do you avoid races in k-means?	Per-thread subtotals and flags; one thread combines; barriers between phases.
Why is your speedup not equal to the number of cores?	Memory bandwidth, barrier waits, thread start-up, and the parts that stay sequential (Amdahl's law).
How does your server avoid zombies? Zombie vs orphan?	SIGCHLD + `waitpid(WNOHANG)` loop. A zombie is dead but not yet reaped; an orphan is alive and its parent has died (it is adopted by init).
What do `setsid()` and printing the PID achieve?	They detach the process from the terminal; the PID is how you stop it later (`kill`).
fork vs poll vs epoll — advantages and drawbacks?	Section 18 comparison table.
How does the receiver know where a message ends?	Length-prefixed header; `recv_all` loops over partial reads.
What happens if a client disconnects mid-transfer?	`send` fails (no SIGPIPE thanks to `MSG_NOSIGNAL`/ignore), state is freed, server continues.
How do you guarantee that no file is overwritten?	Per-client, per-type counters on the server; `O_EXCL` + suffix on the client.

Demo tips

Prepare a two-minute architecture overview (Figure 1 and Figure 2 work well). Arrange terminals in advance. Keep the test commands from section 20 in a text file ready to paste. Rehearse once with each partner driving.

22. Unclear points in the handout — and safe choices

What you will notice	Safe way to handle it
The title says "Lab 2" but the page headers say "Lab 3, 2022 fall".	Same document; the header is a template leftover. Follow the content.
The server is started as `./server -p 9999` in one example and `./server 9999` in another.	Implement `-p` (required). Optionally also accept a bare port number.
Source files are `matinv-par.c` / `kmeans-par.c`, but client commands are `matinvpar` / `kmeanspar`.	Use the file names for the sources and the command names in the protocol, exactly as shown.
The client executable is supposed to be in the project folder, which already contains a `client/` directory.	Impossible as written (section 9). Build `client/client` or choose another name; confirm with the TA.
"Ubuntu's latest LTS", while the provided code was tested on 22.04.	Test on the exact machines or Ubuntu version used for grading; avoid non-portable features.
The mux strategies must stay one process, but a computation can take seconds.	Choose inline compute or a worker thread (section 17), explain it, and confirm with the TA.
"The client folder shall contain all files related to the client only", but the client needs the protocol definitions.	Keep a small copy of the protocol header in `client/` so the client folder is self-contained, or document the shared include.
Unsupported options must return 3.	Also apply this to strategies you did not implement (e.g. `-s muxscale` if you stop at grade C).
Should binaries and object files be submitted?	Ask; normally submit sources after `make clean`.

23. Final submission checklist

- ☐ Folder tree matches section 1.1 exactly (names, nesting, headers in `include/`, sources in `src/`).
- ☐ `make` from a clean checkout builds everything with zero warnings; `make clean` works.
- ☐ Server options `-h`, `-p`, `-d`, `-s` behave as specified; unsupported → usage + exit code 3.
- ☐ Client accepts `-ip` and `-p`, shows clear feedback for every file sent and received.
- ☐ Parallel matrix inversion: identical output to sequential for many sizes; the algorithm, not just initialisation, is parallel.
- ☐ Parallel k-means (grade C+): identical results; `-f` and `-k` work; the data file is uploaded by the client.
- ☐ Results always written to `computed_results/`, sent back, saved in `results/`, never overwritten.
- ☐ fork strategy: concurrent clients, no zombies under load, server runs until Ctrl+C.
- ☐ Daemon (`-d`, grade C+): prints PID, detached, works after the terminal closes, stops with `kill`.
- ☐ muxbasic (grade B+) and muxscale (grade A): one process, many clients, scripted load test passes.
- ☐ Sanitisers and valgrind are clean; no memory growth over many requests.
- ☐ Both partners can explain every file, every design choice, and its drawbacks.

Appendix — Glossary and man pages

Term	Meaning
Barrier	A meeting point: every thread waits there until all threads have arrived (`pthread_barrier_wait`).
Centroid	The centre (average position) of a cluster in k-means.
Data race	Two threads touching the same memory at the same time, at least one writing, without coordination.
File descriptor (fd)	A small integer the kernel gives you for an open file or socket.
Non-blocking I/O	Calls return immediately with `EAGAIN` instead of waiting when no data or buffer space is available.
PID	Process ID, the number that identifies a running process.
Pivot	In Gauss–Jordan, the diagonal element used to clear its column in all other rows.
Re-entrant	A function that can run several times at once safely, because it keeps no shared hidden state.
Speedup	Sequential time ÷ parallel time.

Man pages worth reading: `man 2 fork`, `man 2 waitpid`, `man 7 signal`, `man 2 sigaction`, `man 3 pthread_create`, `man 3 pthread_barrier_wait`, `man 2 socket`, `man 2 bind`, `man 2 listen`, `man 2 accept`, `man 2 connect`, `man 7 tcp`, `man 2 poll`, `man 2 select`, `man 7 epoll`, `man 2 setsid`, `man 3 daemon`, `man 3 syslog`, `man 3 getopt`, `man 1 ps`.